Rechtsklick & Ctrl+C/V — Multi-Agent-Analyse

AgenticExplorer v1.6.26 — Build-Test User-Findings

4-Agent-Team · Bug-Fix-Pipeline Phase 1 · 2026-04-24

> User-Pain im frischen Build:
>
> 1. Rechtsklick zeigt erst das WebView2-Page-Menü (Zurück/Aktualisieren), erst nach mehrmaligem Klick erscheint das Editor-Menü mit Kopieren.
> 2. Ctrl+C kopiert nicht zuverlässig. Ctrl+V geht gar nicht — nur Shift+Ctrl+V.

Problem 1 — Rechtsklick

Beobachtung (User-Screenshot):

Klick	Menü	Inhalt
1. Rechtsklick auf Selektion	**WebView2 Page-Default**	Zurück · Aktualisieren · Speichern unter · Drucken · Weitere Tools
2./3. Rechtsklick spammen	**WebView2 Editor-Default**	Emoji · Ausschneiden · Kopieren · Einfügen · Alles auswählen

→ Die User braucht Glück bzw. mehrere Klicks bis er kopieren kann. Nervig.

Problem 2 — Ctrl+C / Ctrl+V

Native Windows PowerShell: Ctrl+C kopiert (oder SIGINT), Ctrl+V fügt ein. ✅

In AgenticExplorer:

- **Ctrl+C** kopiert nicht zuverlässig → User greift zur Maus
- **Ctrl+V** funktioniert nicht — nichts passiert
- **Shift+Ctrl+V** funktioniert (Linux-Style Paste)

→ User muss Tastenkombinationen lernen, die er sonst nirgends braucht. Akzeptabel? Nein.

Agent-Team

Agent	Layer	Fokus
D1	xterm-Selection-Mechanik	Canvas-Overlay vs. DOM-Selection, Helper-Textarea, Custom-Key-Handler-Logik
D2	Tauri/WebView2 + Clipboard	Default-Kontextmenü-Suppression, `tauri-plugin-clipboard-manager`, Capabilities, CSP
D3	Best-Practice-Vergleich	VS Code, Wave Terminal, Hyper, Tabby, Windows Terminal — wie machen die das?
D4	Code-Audit	Unser `attachCustomKeyEventHandler`, Edge-Cases, Test-Coverage, Race-Conditions

Ultrathink, parallel dispatched, je < 2500 Wörter.

D1 — Kernbefund: xterm-Selection ist Canvas, nicht DOM

Zentrale Erkenntnis (Confidence 95%):

`@xterm/xterm@6.0.0` rendert die Markierung als **Canvas-/DOM-Overlay**, nicht als echte `window.getSelection()`-DOM-Selection.

API-Aufruf	Was er macht
`term.getSelection(): string`	xterm-internes Datenmodell — KEINE DOM-Selection
`term.hasSelection(): boolean`	Same
`window.getSelection()`	leer / ignoriert die xterm-Markierung

Folge für WebView2:

WebView2 unterscheidet beim Rechtsklick:

1. „Klick auf Bereich mit DOM-Selection" → Editor-Menü
2. „Klick auf editierbares Element (textarea, input)" → Editor-Menü
3. „Klick auf nicht-editierbaren Bereich" → **Page-Default-Menü** (Zurück/Aktualisieren/...)

xterm-Canvas fällt in Kategorie 3 → Image 1.

D1 — Warum erscheint dann das Editor-Menü später?

xterm rendert intern ein verstecktes `<textarea class="xterm-helper-textarea">` als Input-Surrogate für Tastatureingaben.

Ablauf beim mehrfachen Rechtsklick:

```
Klick 1  →  Maus auf Canvas-Overlay
         →  WebView2 sieht: nicht-editable
         →  Page-Default-Menü (Image 1)

Klick 2/3 →  Fokus rutscht auf xterm-helper-textarea
          →  WebView2 sieht: editable
          →  Editor-Menü (Image 2)
```

→ **Beide Images sind 1:1 erklärbar durch dieses Modell.**

xterm.js v6 hat KEINE Built-in-Option, das Selection-Overlay als DOM-Selection zu rendern. Auch `screenReaderMode` ändert das nicht grundlegend.

D2 — Tauri-Config-Audit

Stand der App (verifiziert):

Aspekt	Wert	Konsequenz
Tauri-Version	v2	Neue Permissions/Plugin-Architektur
`tauri-plugin-clipboard-manager`	**NICHT installiert**	`navigator.clipboard` ist alleinige Brücke — fragil
`capabilities/*.json`	dialog, shell, updater, fs etc. — **keine clipboard-Permissions**	Selbst mit Plugin müsste hier was rein
`@xterm/addon-clipboard`	NICHT geladen	xterm hat keine optimierte Clipboard-Bridge
`tauri.conf.json` Context-Menu	KEIN Eintrag	WebView2 zeigt sein Default
Frontend `contextmenu`-Handler	KEIN globaler `preventDefault`	Default-Menü greift
CSP	`default-src 'self'` etc.	Beeinflusst Clipboard nur indirekt

Tauri v2 hat KEINE deklarative Config zum Deaktivieren des Default-Kontextmenüs. Lösung muss JS oder Rust-Hook sein.

D2 — Tauri-Plugin-Clipboard-Manager: was fehlt

Was wir aktuell tun (`SessionTerminal.tsx:111`):

```
navigator.clipboard.writeText(selection).catch(err => logError(...));
```

Probleme dieser Brücke in Tauri-WebView2:

1. WebView2 verlangt Secure-Context + User-Geste — Tauri-Origin `tauri://localhost` ist Secure, aber Clipboard-API kann inkonsistent sein
2. **Silent Failure**: `.catch(err => logError())` schreibt nur in eine Logdatei → User sieht nichts
3. Lese-Pfad (`navigator.clipboard.readText()`) ist noch restriktiver — oft blockiert

Offizielle Tauri-v2-Lösung:

```
npm run tauri add clipboard-manager
```

Frontend dann:

```
import { writeText, readText } from '@tauri-apps/plugin-clipboard-manager';
await writeText(selection);
const pasted = await readText();
```

- Permissions in `capabilities/default.json`:
  `clipboard-manager:allow-read-text`, `clipboard-manager:allow-write-text`

D3 — Wie machen es andere Apps?

App	Rechtsklick	Ctrl+C	Ctrl+V	Quelle
VS Code Terminal	Custom Context-Menu (DOM)	Copy if selection / SIGINT	Paste from clipboard	`terminalContextMenu.ts`, Setting `terminal.integrated.rightClickBehavior: "copyPaste"`
Wave Terminal (Tauri)	Custom React-DOM-Menu	Interrupt (kein Auto-Copy)	xterm Default	wavetermdev/waveterm
Tabby (Electron)	Custom + Issue über Konflikt	Interrupt	Paste	Eugeny/tabby
Windows Terminal	Native ConHost-Menü	Interrupt (Setting-bar)	Paste (seit 2021)	microsoft/terminal
xterm.js raw	KEINS (Browser-Default)	→ PTY (SIGINT)	Shift+Insert oder Shift+Ctrl+V	xtermjs/xterm.js
Native ConHost (cmd.exe)	Browser-Default + Paste-on-RClick	SIGINT (NIE Copy!)	NICHTS (Backward-compat)	Windows

→ **State-of-the-Art:** VS-Code-Stil = Custom-DOM-Menü + OS-aware-Keybinds.

D3 — Konvention: was erwartet ein Windows-User?

Windows Terminal 2021+ ist die maßgebliche moderne Konvention:

Taste	Verhalten
Ctrl+C	Copy if selection, SIGINT otherwise
Ctrl+V	Paste aus System-Clipboard
Ctrl+Shift+V	Paste-as-plain-text (Fallback)
Rechtsklick	Custom-Menü (Copy/Paste/Clear/Search)

Alte Linux-Konvention (xterm-default):

- Ctrl+Shift+C/V — weil Ctrl+C/V für Shell reserviert war

Unser Setup übernimmt die Linux-Konvention by default. Das ist der gefühlte Bug — wir sehen aus wie ein Linux-Tool, obwohl wir auf Windows laufen.

D4 — Code-Audit: Custom-Key-Handler

`SessionTerminal.tsx:106-120`:

```
term.attachCustomKeyEventHandler((event: KeyboardEvent): boolean => {
  if (event.type !== "keydown") return true;
  if (event.ctrlKey && event.key === "c") {
    const selection = term.getSelection();
    if (selection) {
      navigator.clipboard.writeText(selection).catch(err => logError(...));
      return false; // Consumed
    }
    return true; // SIGINT
  }
  return true;
});
```

Bewertung:

- ✅ Strukturell richtig für reine Ctrl+C
- ⚠️ `event.key === "c"` ist case-sensitive — Shift+Ctrl+C wäre `"C"` (greift nicht; aber: Shift+Ctrl+C ist ohnehin selten)
- ❌ **Kein Ctrl+V-Handler** (bewusst nicht — Kommentar Z. 88-92)
- ❌ Kein Toast bei Clipboard-Failure
- ❌ Kein Fokus-Schutz: User markiert → klickt Sidebar → Selection bleibt visuell, Fokus weg → Ctrl+C-Event kommt nicht beim Terminal an

D4 — Der Ctrl+V-Mythos

Im Code-Kommentar (Zeile 102–105):

> *„Ctrl+V is intentionally NOT handled here — xterm's native paste handler fires on DOM `paste` events independently of this keydown handler."*

Das stimmt so nicht. Verifiziert via xterm.js v6 Source:

Ereignis	xterm-Verhalten v6
User drückt **Ctrl+V**	Wird als raw `\x16` (SYN-Code) an `onData` weitergeleitet → an PTY → Shell sieht ein literal `^V`
User drückt **Shift+Ctrl+V**	xterm löst intern einen DOM-`paste`-Event aus → Clipboard-Read → `term.paste(text)`
Browser-`paste`-Event (z. B. via Browser-Menü)	xterm behandelt korrekt

→ **Ctrl+V geht nicht „durch xterms native handler", sondern wird als Junk an die PTY geschickt.** Das ist der Bug.

Daher Shift+Ctrl+V funktioniert (xterm-internal Paste-Event-Trigger), Ctrl+V nicht.

SYNTHESE — Bug 1 (Rechtsklick) erklärt

```
User markiert Text mit Maus
        │
        ▼
xterm rendert Selection als Canvas-Overlay (KEINE DOM-Selection)
        │
        ▼
User Rechtsklick auf den markierten Bereich
        │
        ▼
WebView2 prüft: ist das ein editable Element oder hat es DOM-Selection?
        │
        ├── nein  →  Page-Default-Menü (Image 1: Zurück/Aktualisieren/...)
        │
        ▼  (mehrfach klicken / klicken auf andere Stelle)
xterm-helper-textarea bekommt Fokus
        │
        ▼
WebView2 prüft erneut → editable! → Editor-Menü (Image 2)
```

→ Single Root Cause: WebView2 Default-Kontextmenü greift, weil wir es nie deaktiviert/überschrieben haben.

SYNTHESE — Bug 2 (Ctrl+C/V) erklärt

```
Ctrl+C    →  attachCustomKeyEventHandler-Pfad
          ├── Selection vorhanden → navigator.clipboard.writeText (silent-fail-fähig)
          └── Selection leer      → \x03 → PTY → SIGINT  ✓

Ctrl+V    →  attachCustomKeyEventHandler-Pfad
          └── kein Match → return true → \x16 (SYN) an PTY → Shell ignoriert / zeigt ^V  ✗

Shift+Ctrl+V → xterm internal handler → DOM-paste-Event → term.paste(text) → onData → PTY  ✓
```

Zwei separate Probleme:

1. **Ctrl+V ist faktisch nicht implementiert.** xterms „nativer Paste-Handler" deckt nur Shift+Ctrl+V ab.
2. **Ctrl+C `navigator.clipboard.writeText` kann silent failen.** User merkt's nicht, glaubt es ist kaputt.

Layered Defect Map

```
┌──────────────────────────────────────────────────────────────┐
│ LAYER 4 — User-Erwartung                                     │
│   Windows-Konvention (Ctrl+C/V wie in Windows Terminal)      │
├──────────────────────────────────────────────────────────────┤
│ LAYER 3 — App-UX-Bridge (FEHLT)                              │
│   ✗ Custom-Kontextmenü                                       │
│   ✗ Custom Ctrl+V-Handler                                    │
│   ✗ Toast bei Clipboard-Failure                              │
├──────────────────────────────────────────────────────────────┤
│ LAYER 2 — Tauri-Plattform (UNVOLLSTÄNDIG)                    │
│   ✗ tauri-plugin-clipboard-manager fehlt                     │
│   ✗ Clipboard-Permissions in capabilities/                   │
│   ✗ Default-Kontextmenü-Suppression                          │
├──────────────────────────────────────────────────────────────┤
│ LAYER 1 — xterm.js v6 (FEATURE-LIMITIERT)                    │
│   - Selection ist Canvas-Overlay (nicht änderbar)            │
│   - Default-Paste nur via Shift+Ctrl+V                       │
│   - `xterm-helper-textarea` Fokus-Surrogate                  │
└──────────────────────────────────────────────────────────────┘
```

→ **Fixes liegen alle in Layer 2 + 3** — Layer 1 bleibt wie es ist.

Ranked Hypothesen

#	Hypothese	Confidence	Layer
H1	xterm-Selection = Canvas → WebView2-Default-Menü greift	95%	1+2
H2	Mehrfach-Klick fokussiert helper-textarea → Editor-Menü erst dann	90%	1
H3	Kein Custom-Ctrl+V-Handler → `\x16` an PTY statt Paste	90%	3
H4	`navigator.clipboard` ohne Tauri-Plugin = silent fail-fähig	80%	2
H5	User markiert + klickt anderswo → Fokus weg → Ctrl+C trifft nicht Terminal	70%	3
H6	`event.key === "c"` (klein) ignoriert Shift+Ctrl+C-Variante	50%	3 — Edge-Case
H7	CSP `clipboard-read` nicht deklariert blockiert readText	40%	2 — sekundär

Anti-Hypothesen

Was es **nicht** ist:

- ❌ **xterm.js-Bug** — alle Verhalten sind dokumentiert / by design für Linux-Konvention
- ❌ **Tauri-CSP-Block** — CSP betrifft Network, nicht Clipboard direkt
- ❌ **Ctrl+V wird von WebView2 abgefangen** — kein Beleg, das wäre OS-spezifisch und auch in Windows-Apps unüblich
- ❌ **„Shift+Ctrl+V ist Windows-Standard"** — falsch, das ist Linux/gnome-terminal. Windows-Standard ist Ctrl+V (Windows Terminal seit 2021)
- ❌ **navigator.clipboard.writeText funktioniert immer in Tauri** — funktioniert oft, aber nicht garantiert; offizieller Weg ist das Plugin
- ❌ **Unser Custom-Ctrl+C ist generell kaputt** — der Standard-Pfad funktioniert (Selection da → Copy), nur die UX-Feedback-Schleife fehlt

Fix-Optionen

Option A — Quick-Win (~30 min)

1. **Globaler `contextmenu`-Listener** im Terminal-Container mit `e.preventDefault()` → Default-Menü weg, Selektion + Tastenkombi reicht zum Kopieren.
2. **Custom Ctrl+V-Handler** im `attachCustomKeyEventHandler`: liest Clipboard via `navigator.clipboard.readText()`, sendet an PTY via `wrapInvoke("write_session", { id, data: text })`. De-Dedup-Flag gegen Doppel-Paste mit DOM-Event.

→ Bug 1 + Bug 2 weg. Kosmetisch noch ohne Custom-Menü, aber ergonomisch.

Fix-Optionen

Option B — Robust + offiziell (~2 h)

Option A +

1. **`tauri-plugin-clipboard-manager` installieren** (`npm run tauri add clipboard-manager`)
2. Frontend statt `navigator.clipboard.*` → `@tauri-apps/plugin-clipboard-manager` `writeText` / `readText`
3. **Permissions in `capabilities/default.json`**: `clipboard-manager:allow-read-text`, `clipboard-manager:allow-write-text`
4. Toast-Notification bei Clipboard-Failure (sichtbar für User)

→ Robuste, offizielle Tauri-v2-Lösung. Future-proof.

Fix-Optionen

Option C — VS-Code-Stil (~3-4 h)

Option B +

1. **Custom React-Kontextmenü** (DOM-Komponente, auf `onContextMenu` triggern)
   - Buttons: Copy · Paste · Clear · Select All · Search
   - Disabled-State: „Copy" greift nur bei `term.hasSelection()`
2. **Setting in App-Settings**: `terminal.rightClickBehavior: "menu" | "copyPaste" | "default"` analog VS Code
3. **Regression-Tests** für: rightClick-Suppression, Ctrl+V-Handler, Plugin-Integration, Toast-Trigger

→ Polished UX wie VS Code Terminal. Höhere Investition, aber maximaler User-Komfort.

Empfehlung & Priorität

Empfehlung: Option B als guter Mittelweg.

	Quick (A)	Robust (B)	Polished (C)
Aufwand	30 min	2 h	3-4 h
Bug 1 weg	✅	✅	✅
Bug 2 weg	✅	✅	✅
Silent-Failures sichtbar	❌	✅	✅
Future-proof (Tauri-API)	⚠️ navigator.clipboard fragil	✅	✅
Custom-Menü mit „Suchen" etc.	❌	❌	✅
Risiko	niedrig	niedrig	mittel (mehr Code)

Begründung Option B:

- Closes both bugs cleanly
- Nutzt offizielle Tauri-API → keine Surprise-Failures bei Updates
- Setzt Foundation für Option C falls später gewünscht
- 2 h sind angemessen für die Schmerzlinderung

Implementierungs-Reihenfolge (Option B)

F1 — Plugin + Permissions (Phase 2 Step 1):

1. `npm run tauri add clipboard-manager` — Cargo + npm + lib.rs Init
2. `capabilities/default.json` ergänzen
3. Smoke-Test dass `writeText` aus Frontend funktioniert
4. Commit: `chore(tauri): add clipboard-manager plugin`

F2 — Frontend-Integration (Phase 2 Step 2):

1. `SessionTerminal.tsx`: `navigator.clipboard.writeText` → Plugin
2. Custom Ctrl+V-Handler hinzufügen mit Plugin-`readText`
3. `contextmenu`-Listener mit `preventDefault` auf Container
4. Toast-Mechanismus bei Clipboard-Failure
5. Commit: `fix(ui): native clipboard + ctrl-v handler + suppress default context menu`

F3 — Tests & QA (Phase 2 Step 3):

1. Unit-Tests Plugin-Mock
2. Integration-Test: Ctrl+V triggert PTY-Input
3. Test: contextmenu-Event preventDefault feuert
4. Commit: `test(sessions): regression coverage for clipboard + context-menu`

Risiken / Trade-offs

Risiko	Mitigation
Plugin-Install bricht den Build	Auf Branch ausprobieren, vor Commit `cargo check` + `npm run build`
Permissions zu eng → Plugin throws	Mit minimalen Permissions starten, dann erweitern
`contextmenu` preventDefault zu breit (auch außerhalb Terminal)	Listener nur auf `containerRef.current` nicht document
Custom Ctrl+V doppelt-paste mit altem DOM-paste-Pfad	De-Dedup-Flag mit Timestamp (50ms-Window)
User benutzt Linux-Setup mit gnome-terminal-Konventionen	Beides supporten (Ctrl+V und Shift+Ctrl+V)
Capability-File-Konflikt mit anderen Permissions	Separate Capability-File `clipboard.json` möglich

Test-Matrix für Phase 3 (User-Verifikation)

#	Szenario	Erwartung
T-RC1	Rechtsklick auf markierten Text	NICHTS oder eigenes Menü — kein WebView2-Default-Menü
T-RC2	Rechtsklick auf leeren Bereich	Same
T-CC1	Ctrl+C mit Selektion	Selektion im Clipboard, sichtbares Feedback (z. B. Selektion verschwindet)
T-CC2	Ctrl+C ohne Selektion	SIGINT — Claude bricht ab
T-CV1	Ctrl+V mit Clipboard-Text	Text erscheint am Prompt, NICHT als `^V`
T-CV2	Ctrl+V mit leerem Clipboard	Nichts passiert (oder leiser No-Op)
T-SCV	Shift+Ctrl+V	Funktioniert weiter wie vorher (Linux-Compat)
T-FAIL	Clipboard-Permission entzogen (manuell testen?)	Toast: „Kopieren fehlgeschlagen: <reason>"

Wartepunkt — User-Freigabe nötig

Phase 1 abgeschlossen:

- 4 Agenten deployed, Findings synthesiert
- 2 Bugs auf Single-Root-Causes reduziert (Layer 2+3)
- 3 Fix-Optionen mit Trade-off-Tabelle

Warte auf:

- ✅ **„Go Option B"** (empfohlen) — Plugin + Frontend + Tests, ~2 h
- 🟡 **„Go Option A"** — Quick-Win mit `navigator.clipboard`, ~30 min, fragiler
- 💎 **„Go Option C"** — VS-Code-Stil mit Custom-Kontextmenü, ~3-4 h
- 🔄 **„Erst Option A testen, danach entscheiden"** — Quick-Win, dann re-evaluieren
- ❌ **„Feedback / Andere Layer vertiefen"**

Quelle dieser Slides: `reports/clipboard-rightclick-multi-agent-analyse-v1.6.26.{md,html,pdf,pptx}`

Keine Code-Änderungen ohne Freigabe — bug-fix-pipeline Phase-Gate.

Anhang · Quellen

- Tauri v2 Clipboard Plugin
- Tauri Discussion #11808 — Disable Context Menu
- Microsoft WebView2 Context Menus Doc
- xterm.js Copy/Paste Doc
- @xterm/addon-clipboard NPM
- VS Code Terminal Right-Click Behavior
- xterm.js Issue #292 — Alt. Copy/Paste Shortcuts
- Windows Terminal Issue #3058 — Ctrl+C/V
- Wave Terminal (Tauri)

Anhang · Interne Code-Referenzen

Datei	Zeile	Thema
`src/components/sessions/SessionTerminal.tsx`	88–92	Comment „Ctrl+V intentionally NOT handled"
`src/components/sessions/SessionTerminal.tsx`	93–107	`attachCustomKeyEventHandler` für Ctrl+C
`src/components/sessions/SessionTerminal.tsx`	111	`navigator.clipboard.writeText`
`src-tauri/tauri.conf.json`	—	Keine Context-Menu-Config
`src-tauri/capabilities/default.json`	—	Keine Clipboard-Permissions
`src-tauri/Cargo.toml`	—	Kein `tauri-plugin-clipboard-manager`
`package.json`	—	Kein `@tauri-apps/plugin-clipboard-manager`
`src-tauri/src/lib.rs`	—	Plugin-Init-Block (hier kommt der neue Plugin-Eintrag rein)
`src/components/sessions/SessionTerminal.test.tsx`	201–243	bestehende Ctrl+C-Tests, kein Ctrl+V

ENDE PHASE 1

Status: Analyse fertig. Marp-Deck in `reports/clipboard-rightclick-multi-agent-analyse-v1.6.26.*`.

Nächster Schritt: User-Entscheidung Option A / B / C.

> „Two clean root causes — both in Layer 2 (Tauri-Config) und Layer 3 (App-UX-Bridge). Layer 1 (xterm) bleibt unangetastet."

— Bug-Fix-Pipeline Phase 1 Gate · warte auf Freigabe.